Data Structures (ETCCDS202)

Assignment - 2

Name of the School: School of Engineering & Technology

Program: B. Tech CSE(AI & ML)

Course Title: Data Structures

Course Code: ETCCDS202

Unit Title: Linear Data Structures

Student Name: Abhay Kumar

Roll No.: 2501730185

Section: A

1. Amortized Analysis of append()

In a dynamic array, the append() operation typically runs in constant time **O(1)**. However, when the array reaches its capacity, it must be resized—usually by allocating a new array with double the capacity and copying all existing elements. This resizing operation takes **O(n)** time.

Despite this occasional expensive operation, resizing occurs infrequently. When analyzed over a sequence of insertions, the total cost is spread across all operations. Hence, the **amortized time complexity** of append() remains **O(1)**.

2. Stack Operations

A stack is a linear data structure that follows the **Last In, First Out (LIFO)** principle. The primary operations are:

- **Push**: Inserts an element at the top of the stack
- **Pop**: Removes the top element
- **Peek (Top)**: Returns the top element without removing it

All these operations execute in **O(1)** time, as they involve only the top element and do not require traversal.

3. Queue Operations

A queue follows the **First In, First Out (FIFO)** principle. The core operations include:

- **Enqueue**: Adds an element to the rear of the queue
- **Dequeue**: Removes an element from the front
- **Front (Peek)**: Retrieves the front element without removing it

By maintaining both **head (front)** and **tail (rear)** pointers, all operations can be performed efficiently in **O(1)** time.

4. Parentheses Checker Using Stack

A stack is an effective data structure for checking whether an expression has balanced parentheses. The algorithm works as follows:

- Traverse the expression from left to right
- Push all opening brackets ((, {, [) onto the stack
- For every closing bracket (), },]), check if it matches the top element of the stack
- If it matches, pop the top element; otherwise, the expression is unbalanced

At the end, if the stack is empty, the expression is balanced.

Examples:

- ([]) → Balanced
- ([)] → Not Balanced
- (((→ Not Balanced
- "" (empty string) → Balanced

5. Conclusion

This assignment explored fundamental linear data structures, including **Dynamic Arrays, Linked Lists, Stacks, and Queues**, along with their implementation and time complexity analysis. Additionally, we applied the stack data structure to solve the balanced parentheses problem.

These concepts form the foundation for efficient data organization and are essential for designing optimized algorithms in computer science.